

Data-Driven RSI: Evaluation, Ladders, and Trajectories

What RSI must first take over is the improvement loop now driven by foundation-model teams: find problems, design evaluations, produce data, train, and choose the next move. This note starts with the part of that loop that seems easiest to test—data.

Eval

Synthetic data

Harness-Evolve

What RSI is meant to take over

Many visions of RSI become the same engineering question: can models gradually take over the improvement loop that foundation-model teams still maintain by hand? Teams at OpenAI, Moonshot AI—the company behind Kimi—and Zhipu AI—the developer of GLM—still rely on researchers to find failures, build evaluations, produce data, run training, and choose the next direction.

RSI is meant to take over work that foundation-model researchers like us still perform. An LLM should be able to locate its capability boundary, propose worthwhile problems, create experience for the next training round, and test whether its successor is genuinely stronger, even when nobody is continuously tending the loop. A research loop that can keep running in this way would be a first step toward RSI.

Why start with data

Data offers a practical starting point. Important algorithmic changes are hard to predict, and promising ideas often disappear at scale. With data, the experiment is concrete: give a model unfamiliar tasks, locate stable failures, turn them into new tasks and trajectories, and train those back into the next model.

The key is to let the model's failures change what gets produced next. Additional tokens are only the medium. If this loop can keep running, the model is already participating in the decision about what it should learn next. That remains short of full RSI, but it may be the first part that can be tested seriously.

Following that line of thought leaves me with three questions:

- How can an open-ended goal such as “be better at mathematics” or “be better at coding” be evaluated well enough to decide what to train next?
- When a batch of synthetic data looks good, how can its absorption by the target model be measured, and how much should be produced?
- How can synthesis techniques accumulated across domains become a repeatable production line for useful trajectories?

Those are the three questions considered here.

Working definitions

This note uses a weaker, testable definition of **RSI**: the current model helps identify capability gaps and produce experience for the next training round; the resulting successor then improves reproducibly on fresh tasks outside production. Parameter rewriting during a single inference run sits outside this definition.

Open-ended evaluation allows a broad goal, such as mathematical research ability. Each iteration still needs auditable evidence: tasks, trajectories, failure causes, verifier outcomes, and a data prescription for the next round.

This note uses **Harness—Evolve** as a working name for the production process. The harness defines the environment in which the model operates. Evolve generates, revises, and selects candidate trajectories inside that environment. SFT then trains selected processes into the target model. Weight updates happen during SFT.

Why this leads to synthetic data

Push the data-driven view one step further and synthetic data becomes unavoidable.

Many SFT examples, preference pairs, critiques and rewrites, verifier-filtered rollouts, and tool-use trajectories are constructed through some combination of models, rules, tools, and people. Recent post-training gains often rely on this deliberately constructed experience.

Pretraining is beginning to acquire the same character. Natural data remains the foundation, but once cleaning, deduplication, and filtering improve, more of the marginal data will come from rewriting, transformation, difficulty control, reasoning completion, execution-based filtering, or turning existing material into a task that is actually worth learning. The boundary between pretraining and post-training data will become less clean over time.

Natural data remains the foundation. It gives a model knowledge, language, and contact with the world. It rarely grows around the failure modes of a particular checkpoint. When a model gets stuck on one step of one kind of problem, the web rarely supplies ten thousand exercises at exactly the right difficulty with reliable feedback attached.

The data wall describes a growing gap between demand and useful high-quality data. The next model needs material that is new to it, relevant to its weaknesses, and good enough to train on. Estimates of public text stocks and training demand suggest that high-quality human-generated text can become a constraint on continued scaling (Villalobos et al., 2024). The date of impact is hard to predict, and additional tokens already fail to produce capability reliably.

Will DePue calls the internet a “one-time subsidy” to deep learning (DePue, 2026). For decades, people wrote webpages, code, papers, and discussions with no model-training plan; together they happened to form a huge, inexpensive, cross-domain dataset. The next increment is unlikely to arrive through the same accident. Much of the valuable material outside the public web lives in internal workflows, tacit expert judgment, unrecorded failures, and processes visible only inside real environments.

This separates the data problem into at least two axes. **Volume** asks how many more high-quality examples are needed in domains already represented. **Coverage** asks which tasks, tools, edge cases, and long-horizon processes have never been recorded at all. Synthetic data can expand volume. Coverage still requires anchors from people and real environments, after which models can elaborate, recombine, and explore around them.

The scarce resource is experience that is useful for the model’s next step.

Continued scaling will require models to help create the next round of experience: turn failures into tasks, turn tool and verifier feedback into process, and turn the useful parts of that process back into training data. Synthetic data therefore becomes an important precondition for RSI.

Synthetic data can also copy the past more quickly. A model can generate millions of answers from abilities it already has. If the same model produces, judges, and filters those answers, its preferences can become more concentrated with every round. Evaluation, a data ladder, and a serious trajectory pipeline therefore have to be designed together.

1. Evaluation has to keep moving

The resolution of a fixed benchmark rarely lasts indefinitely. As frontier models approach its ceiling, adapt repeatedly to its task format, or encounter its examples during training, its ability to distinguish the strongest systems can decline. Different benchmarks, however, saturate at different rates and for different reasons. In a study of 60 widely used text benchmarks, nearly half showed high or very high saturation (Akhtar et al., 2026). Contamination makes the score harder to interpret still, with effects that vary by model and benchmark (Singh et al., 2024).

Building a harder benchmark is the usual response, and the process is becoming expensive. Humanity’s Last Exam selected 2,500 questions from more than 70,000 submissions, involving nearly a thousand experts and several stages of review (Center for AI Safety & Scale AI, 2025). This leads to a prediction worth testing: if model iteration keeps accelerating, the human cycle for defining tasks, writing questions, maintaining answers, and refreshing validation sets will eventually fall behind the model’s own improvement cycle.

Evaluation has to enter the self-improvement loop. The goal can stay open-ended—better mathematical research, stronger coding, longer-horizon work—while the model uses the previous failure map to search for its next boundary, propose tasks, construct counterexamples, control difficulty, and continuously grow new benchmarks and validation sets. AutoBench already formulates benchmark creation as an LM-driven search for datasets that are salient, novel, and difficult (Li et al., 2025).

A model can help propose benchmark items, tests, and verifiers. Independent validation still carries the burden of proof. An active benchmark used for diagnosis and training can keep changing. Private validation used to confirm progress should remain isolated from the training pipeline and hidden from the evaluated checkpoint.

Independent generators, environment evidence, or human review check task validity, answer correctness, and leakage. Freezing the checkpoint helps when paired with these checks.

A model can write its next test. Independent evidence confirms the improvement.

Self-evaluation maintains a moving capability boundary. Self-validation lets the model help construct a test, then uses hidden and tamper-resistant evidence to settle the result.

From an open-ended goal to the next training round

A benchmark becomes a process for continually locating the boundary.

01

Name the direction

Mathematics, coding, research, or longer-horizon work.

02

Search the boundary

Propose fresh tasks, counterexamples, and harder variants.

03

Build a failure map

Locate repeatable failures and preserve details beyond the score.

04

Produce data and train

Turn failures into tasks, trajectories, and a successor model.

05

Validate the successor

Compare it on hidden tasks and independent evidence, then feed the result into the next boundary search.

VALIDATION BOUNDARY

Private validation

Hidden from the evaluated checkpoint and strictly separated from training.

Independent evidence

Execution, environment feedback, traceable sources, separate verifiers, and human audits.

Figure 1. Steps 01–05 form one iteration. Validation feeds the next boundary search, while the validation boundary prevents the system from writing the test, training on it, and declaring victory.

An evaluation round should produce a **failure record** that changes what happens next. Keep the score, then record the task slice, terminal error, earliest causal deviation in the trajectory, candidate mechanism, and proposed data intervention. Two runs can both end in a timeout while failing for entirely different reasons—bad planning, a broken tool call, or context-management failure.

Five layers of evidence matter:

1. **Outcome:** Was the task completed under an executable or explicit verifier?
2. **Process:** Where did the first causal deviation occur?
3. **Slice:** Does the mechanism repeat across a fresh task family? One bad case provides weak evidence.
4. **Prescription:** Does the failure imply a concrete task, feedback, or harness intervention?
5. **Transfer:** After training, does the gain appear on private validation that did not participate in data production?

2. Ladders: use small experiments to decide whether to climb

Imagine that a coding harness can produce one million trajectories in a week. The decision to run at full scale needs evidence. Synthetic data can look polished and be fully correct while doing almost nothing when trained into the target model.

A ladder is a **stage-gated experiment**. It breaks one expensive run into increasingly costly rungs, scales one object at a time, holds the other conditions as stable as possible, and climbs only when the signal over a control is repeatable. Scaling laws may emerge later from enough of these experiments.

In architecture scaling, each rung represents a larger compute budget. At every rung, the candidate should be compared with a matched baseline under a preregistered data distribution, objective, evaluation protocol, and rule for allocating parameters and tokens. The question is: **does the advantage over baseline survive scale and earn the next compute budget?**

A data ladder changes the object being scaled. Hold the target model, training configuration, and evaluation fixed; increase the amount or coverage of data from a fixed source; measure what each additional tranche contributes on held-out tasks. It asks: **how much new learning signal remains, and where does marginal return disappear?**

Synthetic data is harder because a production recipe creates the dataset on demand. The producer, harness, verifier, sampling policy, and filter jointly determine its distribution; the target model determines whether those trajectories can be absorbed. A comparable synthetic-data ladder must therefore hold the **production recipe and target model fixed**, varying only the volume of accepted, deduplicated trajectories. A change to the producer, harness, verifier, sampling policy, filter, or target model sends the ladder back to its cheaper rungs for recalibration.

Scaling production can itself change the output: acceptance may fall, duplication may rise, and easy cases may crowd out the long tail. The same trajectories can teach one target model a planning habit and another little more than wording. SynthLLM likewise reports saturation regions that vary with target-model scale (Qin et al., 2025). Useful volume is specific to the production recipe and target model.

All three ladders make small experiments precede large ones, but their experimental units and failure conditions differ:

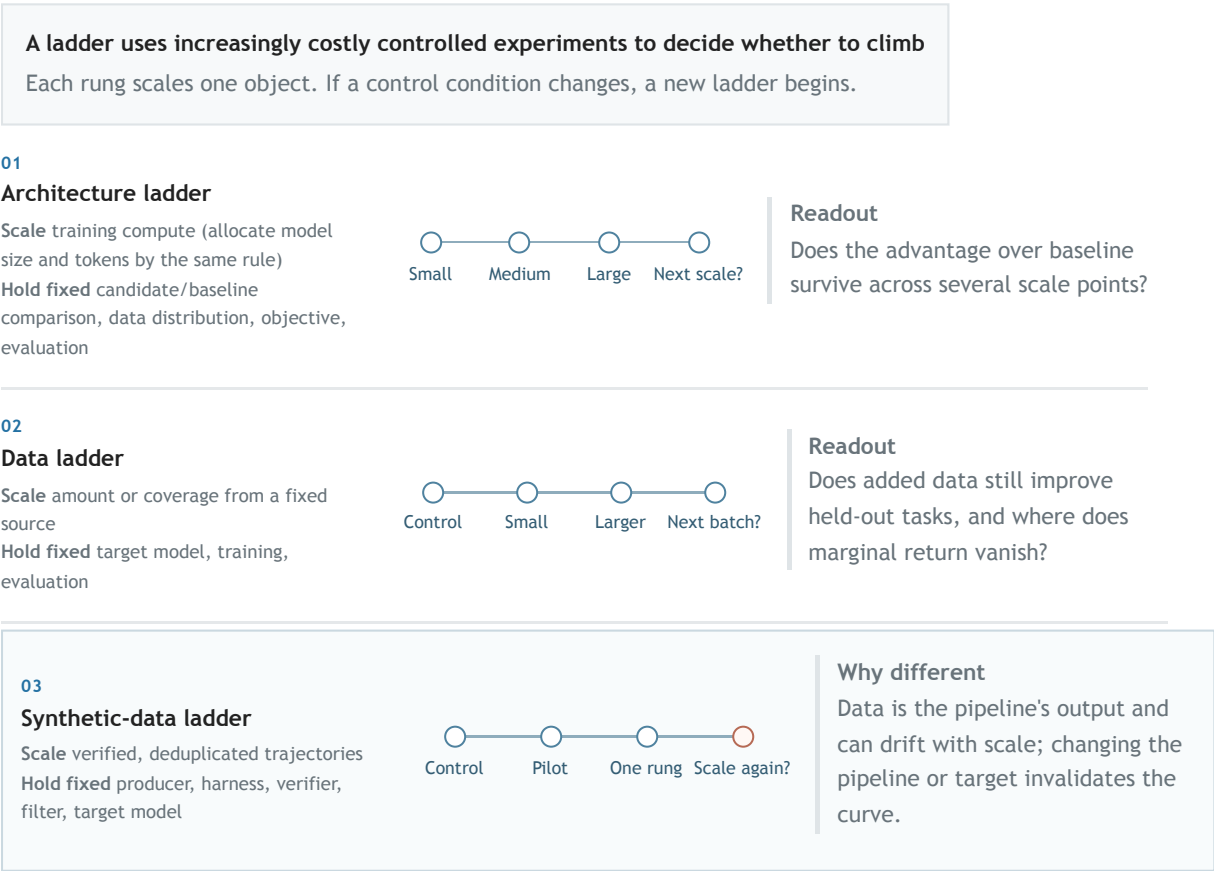


Figure 2. Each synthetic-data ladder belongs to one production recipe and one target model.

A synthetic-data ladder is therefore a sequence of increasingly expensive bets: audit a small set of tasks and verifiers; run a pilot from the same base checkpoint; scale only after repeatable held-out signal; repeat when gain approaches the threshold implied by noise, cost, and regression risk; stop—or redesign the pipeline and begin a new ladder—when it falls below.

A minimum viable experiment

For the ladder to be interpretable, only a small number of variables should move at once. A minimum design fixes the base checkpoint, optimizer, number of epochs—or another preregistered exposure rule—and held-out evaluation, while varying only the number of accepted, deduplicated trajectories; total training tokens then grow with data volume. A fixed training-token budget measures coverage or mixture under equal compute. Pure data scaling lets total training tokens grow with data volume. Report the two experiments separately. Repeat each rung at least twice, and estimate training noise alongside the trend.

Each point needs three groups of measurements: generation cost, acceptance rate, and deduplication rate on the production side; effective training tokens, stability, and behavioral change on the training side; target-slice gain, cross-slice transfer, and regressions on the evaluation side. A more expensive rung is justified only when adjacent rungs show a consistent direction across repeats.

The name **ladder experiment** is more accurate until the curve has been tested across enough scale points, target models, and production pipelines. Existing results have limited reach. The immediate value lies in pacing investment and revealing saturation; a stable scaling law needs more evidence.

At every rung, ask again: how much new capability did the next batch buy?

Each project sets its threshold from uncertainty across repeats, production cost, and regression risk.

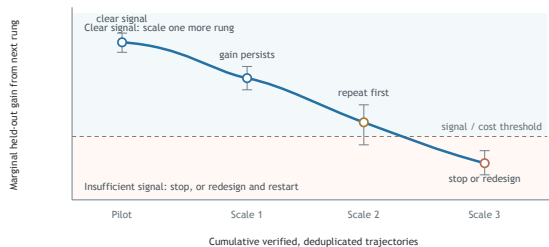


Figure 3. A schematic decision rule. Train every point from the same base checkpoint and repeat it on held-out tasks outside data production; a new production recipe or target model starts a new curve.

What to measure first

Start with three measurements: the share that passes verification, what remains after deduplication, and whether each additional batch of genuinely different trajectories improves an independent evaluation. The first two numbers determine what the production line really costs. The last one tells whether it should keep running. Total generation volume provides context.

Once a small training run shows a signal, accepted and non-duplicate trajectories can be scaled in stages. At each stage, the starting checkpoint, training setup, and evaluation set should remain as stable as possible, with the previous rung retained as a control. If volume grows several times while unfamiliar tasks no longer improve, or if acceptance collapses, the pipeline is approaching saturation.

The right amount of synthetic data lies before saturation. Without a ladder, it is easy to finish producing an expensive dataset and only then begin explaining why it failed.

The ladder belongs to its pipeline and target model

The same synthetic dataset can behave very differently on two models. Every example carries habits from its generator and harness: how problems are decomposed, which tools are preferred, when the model backtracks, and which errors it tends to make. There is no guarantee that another model can absorb those habits.

Whenever the target model changes—or the producer, harness, verifier, sampling policy, or filter is modified—the ladder should return to its cheaper rungs. The previous curve offers useful evidence with limited transfer. This is one reason a scaling law for synthetic data is harder to establish than one for natural data.

3. Trajectories need a production line

Data has always been designed by people. People decide where to look, what is worth keeping, how to label it, and which tasks should come before others. Even when the raw material comes from the internet, it enters a model through a human-built pipeline.

As public webpages stop being the main source of marginal data, production will increasingly shift from collecting documents to recording work. A complete expert workflow can include private tools, repeated communication, local judgment, failure recovery, and final acceptance. Saving only the input and final answer discards much of the trainable process. Data infrastructure must capture environments, actions, feedback, and outcomes together, alongside storage and labeling.

Many synthetic-data pipelines today are essentially prompt + model + filter. They can produce large collections of questions and answers, while leaving the process thin. The useful training signal often lies in how the model searched, tried something, observed an error, revised a plan, and noticed that it had gone wrong.

From a prompt to a production line

In coding, people know that tests matter. In mathematics, they use proof checks, counterexamples, and controlled increases in difficulty. Agent tasks also need environment state and tool output. These pieces of experience currently live in prompts, scripts, and researchers’ heads. A harness turns them into an environment the model encounters every time it works.

Here, a harness covers prompts, tools, context management, workflow, persistent state, permissions, and verification logic: the system that controls how a model observes, acts, stores artifacts, and checks itself (Weng, 2026). In practice, it determines the kind of trajectory the pipeline is able to collect.

Traceable

The task, generator, harness, tools, and verifier all have explicit versions.

Process is visible

Keep the necessary states, actions, tool feedback, failures, and revisions.

Verifiable

Prefer execution or independent judgment, and retain a reason when a trajectory is rejected.

Diverse

Control the mixture of harnesses and generators; remove duplicate paths and repeated phrasing.

Trainable and testable

Send every batch back through the ladder and an independent evaluation.

Harness—Evolve first, then SFT

Put a model in an environment with tools and feedback, and let it make several attempts. It can follow different paths, fail, backtrack, and revise in response to what happens. From those attempts, keep the complete processes that are correct, representative, and meaningfully different from one another.

SFT receives the better trajectories. The training goal is concrete: make reliable behavior easier to reach the next time, verify when verification is needed, revise after failure, and use a tool when the task calls for one.

STaR provides a direct precedent: generate rationales, keep paths that recover the correct answer, and train them back into the model (Zelikman et al., 2022). Harness—Evolve expands the candidate object from a textual rationale to a stateful agent trajectory. Tool responses, file changes, execution errors, backtracking, and replanning become part of the data. DGM and Self-Harness optimize the agent or harness itself. Here the output is training data, and the target model changes in the final SFT stage (Zhang et al., 2025; Zhang et al., 2026).



Figure 4. Four stages of one data pipeline. Evolve improves candidate trajectories; the final SFT stage writes the selected processes back into the target model.

Why use more than one harness

One harness tends to grow one style of trajectory. A test-driven coding harness produces many run—error—revision paths. A critique-and-rewrite harness produces more self-checking. A search-oriented harness produces branches and comparisons. Each can be useful, and each can harden into a template when overused.

A safer design runs several harnesses, then uses the ladder to measure how well the target model absorbs each trajectory source and mixture. The extra experiments reduce the risk of filling the training set with one repeated pattern.

A harness has a horizon too

Long-running tasks place another constraint on the harness. Code written for today’s agents often assumes that a task ends within one run or context, tool calls are synchronous, verifier feedback arrives quickly, and success can be represented by one terminal state. Those assumptions may work for a coding task measured in minutes. Experiments that run for days, data programs spanning multiple training rounds, and research with delayed feedback need a different design.

METR measures a task-completion horizon using the time a human expert needs for the same task. The metric proxies task difficulty and is calculated separately from the agent’s literal wall-clock runtime. Their results isolate an important capability axis: whether a model can reliably compose local skills into a longer sequence of actions (METR, 2026). If task horizon grows faster than harness horizon, the binding constraint may shift to lost state, context growth, compounding errors, and failed recovery.

An RSI-oriented harness needs several capabilities that still lack universal support:

- **Persistent state:** experiments, code, data versions, and unfinished work need storage beyond context; interrupted runs need recoverable checkpoints.
- **Hierarchical goals:** long tasks need verifiable stages that preserve dependencies across stages and resist the pull of the nearest local score.
- **Asynchronous execution:** training, evaluation, and data generation may run for hours or days; the harness must launch, monitor, cancel, and resume background work.
- **Delayed feedback:** when final reward arrives late, intermediate evidence must be retained and failure attributed to the step that caused it.
- **Evolvability with boundaries:** the model may revise workflows, context policy, and tool composition, while verifiers, permissions, and audit logs remain outside the editable loop.

Recent work on long-horizon agents similarly centers compact state, checkpointing, verifier-backed state transitions, and targeted recovery, reducing the repeated use of raw interaction history in the prompt (Wu et al., 2026). This suggests a longer-term extension: Harness–Evolve should evolve trajectories inside the harness and upgrade **the harness carrying those trajectories** as tasks lengthen. Otherwise the production line will reliably generate data only within its current horizon.

Synthesis still needs priors

Goal priors	Know roughly what “better at mathematics” contains while keeping the target broader than one frozen test.
Task priors	Know which tasks are worth producing, how difficulty should increase, and which failures are informative.
Verification priors	Execute what can be executed and check what can be checked; avoid asking the generator to grade itself.
Exploration priors	Allow trial, error, and backtracking while cutting off long trajectories that are going nowhere.
Diversity priors	Control task types, tools, solution paths, and generator sources; keep any one pattern from filling the dataset.

These priors point to useful regions of exploration, trustworthy feedback, and outcomes that need scrutiny. The model also needs to transfer these lessons. Given a new topic or open-ended goal, it should be able to borrow from prior experience, design tasks and environments, and search for the next useful data.

There is also a practical risk: generators copy themselves. If one model acts, judges, and filters, Evolve can concentrate its existing patterns and slow capability growth. Execution tools, independent verifiers, generators from different model families, anchors in real data, and small human audits help keep the production line from becoming an echo chamber.

A second risk is **selection-induced shortcutting**. A verifier that checks only the final answer may select lucky short paths. A preference for long traces can reward empty elaboration. Selection should therefore consider outcome, process completeness, trajectory novelty, and cost together, while retaining rejected traces and rejection

reasons. Those records support cleaning and the next round of task and harness design.

Relationship to adjacent work

The nearest lines of work become easier to separate by their optimization target:

- **Reasoning self-training** optimizes reasoning paths to be internalized by the model. STaR is the clearest example.
- **Synthetic-data scaling** measures the gain and saturation of a generation method as data and model scale change. SynthLLM belongs here.
- **Dynamic evaluation** continually searches for and refreshes benchmark items. AutoBench is one example.
- **Harness optimization** changes prompts, tools, context, or workflow so that a frozen model performs better at deployment. Self-Harness closes a loop among failure mining, editing, and regression testing.
- **Open-ended agent evolution** retains a population of candidate agents and continues branching under executable feedback. DGM and AlphaEvolve show how archives and evaluators support this search ([Novikov et al., 2025](#)).
- **Harness–Evolve in this note** sits at their intersection: use a harness to expand trajectory search, use verifiers and diversity rules to select trajectories, and distill the result into a target model. It remains a research hypothesis awaiting empirical tests.

Three causal tests matter next: whether multiple harnesses produce more transferable data than one; whether a ladder predicts the return of large-scale production; and how much reusable process SFT learns relative to the surface preferences of the generator and verifier.

Putting the loop together

Once the three pieces are connected, the workflow is ordinary: evaluation locates a problem, Harness–Evolve produces trajectories, SFT trains them back into the model, and the ladder decides whether that data method is still worth scaling.

The failures of one model determine the data for the next. As the model improves, old tasks stop being useful and old harnesses saturate. The system keeps changing its tasks, environments, and data. Data itself enters the iteration loop.

01

Name the direction

The goal can begin broad and still be clear enough to explore.

02

Build fresh tasks

Keep changing the test until it reaches failures that matter for the current model.

03

Design the harness

Put human domain knowledge into tools, feedback, verification, and exploration rules.

04

Produce trajectories

Let models try, fail, and revise; retain several kinds of complete process.

05

Filter, then SFT

Train only trajectories that are reliable and meaningfully different.

06

Run the ladder

Measure absorption, transfer, and saturation before deciding to produce more.

The loop earns further study when a model can help identify its failures, produce the next training experience, and become reliably stronger because of that experience.

The goal, harness, verifier, and stopping rule still depend heavily on human judgment. We can make one segment measurable, scalable, and falsifiable now, then leave full autonomy for later.

Open questions

- How can an open-ended goal be covered well enough? How do we keep it from quietly turning into another fixed benchmark?
- How can model-generated benchmarks and validation sets remain novel without mistaking the generator's preferences for the capability boundary?
- How should trajectories from different harnesses be deduplicated and mixed without cancelling one another out?
- When should the data keep scaling, and when should the generator or harness be replaced instead?
- Can the model gradually participate in harness design without causing evaluation and production to collapse into the same loop?
- Does SFT learn a better process, or merely a subtler version of the generator's style?

These questions remain open. The next step is to choose one narrow domain, run the entire loop several times, and record where it breaks.

References

1. Villalobos et al. (2024), *Will we run out of data? Limits of LLM scaling based on human-generated data*.
2. Singh et al. (2024), *Evaluation data contamination in LLMs: how do we measure it and (when) does it matter?*.
3. Zelikman et al. (2022), *STaR: Bootstrapping Reasoning With Reasoning*.

4. Qin et al. (2025), *Scaling Laws of Synthetic Data for Language Models*.
5. Zhang et al. (2025), *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*.
6. Novikov et al. (2025), *AlphaEvolve: A coding agent for scientific and algorithmic discovery*.
7. Zhang et al. (2026), *Self-Harness: Harnesses That Improve Themselves*.
8. Weng (2026), *Harness Engineering for Self-Improvement*.
9. DePue (2026), *A Stargate for Data*.
10. METR (2026), *Task-Completion Time Horizons of Frontier AI Models*.
11. Wu et al. (2026), *StructAgent: Harness Long-horizon Digital Agents with Unified Causal Structure*.
12. Akhtar et al. (2026), *When AI Benchmarks Plateau*.
13. Center for AI Safety & Scale AI (2025), *Humanity's Last Exam*.
14. Li et al. (2025), *AutoBench: Creating Salient, Novel, Difficult Datasets for Language Models*.